



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE - UFRN
CENTRO DE ENSINO SUPERIOR DO SERIDÓ - CERES
DEPARTAMENTO DE COMPUTAÇÃO E TECNOLOGIA DCT



JOSÉ MOISÉS LOPES DA SILVA

**DESENVOLVIMENTO DE UM AGENTE
ESPECIALISTA EM PSICOLOGIA EDUCACIONAL:
IMPLEMENTAÇÃO DE RAG E FINE-TUNING
EFICIENTE VIA LoRA**

Caicó - RN

2026

JOSÉ MOISÉS LOPES DA SILVA

**DESENVOLVIMENTO DE UM AGENTE
ESPECIALISTA EM PSICOLOGIA EDUCACIONAL:
IMPLEMENTAÇÃO DE RAG E FINE-TUNING
EFICIENTE VIA LoRA**

Relatório de atividades apresentado como requisito para obtenção de nota na disciplina **Tópicos Avançados de Inteligência Artificial A**, ofertada pela UFRN no período letivo 2026.1.

Professor: Prof. Dr. Thommas K. S. Flores

Universidade Federal do Rio Grande do Norte - UFRN
Centro de Ensino Superior do Seridó - CERES
Departamento de Computação e Tecnologia - DCT

Caicó - RN
2026

Sumário

1	Introdução	
1.1	Objetivos	
2	Neurônio Artificial à especialização via LoRA	
2.1	Arquitetura dos modelos de linguagem	
2.2	Otimização via Gradiente	
3	Avaliação de Performance e Métricas de NLP	
3.1	Métricas Lexicais: BLEU e ROUGE	
3.2	Similaridade de Jaccard e Fidelidade	
4	Justificativa Técnica do Hiperparâmetro de Rank (r)	
4.1	Análise do Trade-off	
4.2	Reflexão sobre a Especialização no Domínio	
5	Análise de Latência e Eficiência Operacional	
6	Justificativa dos Hiperparâmetros ($BLEU, ROUGE, PPL$)	
7	Arquitetura de Deploy	
8	Aplicação com API funcionando com respostas direcionadas ao cliente	
9	Considerações Finais	

Lista de Figuras

- 1 Representação detalhada do Neurônio Artificial e da Especialização via LoRA.
- 2 Fluxograma básico do procedimento de curadoria manual dos dados.
- 3 Diagrama de Sequência completo da aplicação com conexão com a API
- 4 Gráfico de Convergência.
- 5 Gráfico de Convergência.
- 6 Gráfico de Convergência do modelo Qwen.
- 7 Gráfico de Convergência do modelo GPT-NEO.
- 8 Gráfico Radar dos modelos utilizados no Fine-Tuning
- 9 Resposta do Qwen 0.5B à pergunta do usuário.
- 10 Resposta do Flan T5 Small à pergunta do usuário.
- 11 Resposta do Bart Tiny à pergunta do usuário.
- 12 Resposta do GPT-Neo-125M à pergunta do usuário.
- 13 API funcionando

1 Introdução

A disseminação de sistemas de Recuperação Aumentada por Geração (RAG) tem transformado a interação entre modelos de linguagem e bases de conhecimento proprietárias, mitigando alucinações e provendo respostas fundamentadas em documentos específicos. Contudo, a aplicação genérica destes modelos encontra limitações em domínios técnicos especializados, onde a terminologia e as nuances contextuais exigem uma adaptação mais profunda do que a simples recuperação vetorial permite .

Neste contexto, este trabalho explora o ciclo de vida completo de um sistema de RAG integrado ao ajuste fino eficiente via *Low-Rank Adaptation* (LoRA). A proposta visa transcender a arquitetura de busca semântica tradicional através da especialização de pesos, permitindo que modelos de linguagem ajustem seu comportamento ao domínio específico definido pelo documento base utilizado.

A metodologia empregada contempla quatro etapas fundamentais: (i) a curadoria e geração de um *dataset* de instrução a partir de extração de texto via *chunking*; (ii) a aplicação da técnica LoRA para reduzir o custo computacional do treinamento mantendo a capacidade de adaptação; (iii) a avaliação quantitativa utilizando métricas consagradas de processamento de linguagem natural (NLP), como BLEU, ROUGE e Similaridade de Jaccard; e (iv) a disponibilização do sistema via API RESTful. Este projeto busca demonstrar a viabilidade técnica da especialização de modelos em ambientes de recursos limitados, discutindo os *trade-offs* entre capacidade de adaptação e custo de inferência, validando a eficácia da técnica LoRA na preservação do conhecimento pré-treinado frente à nova especialização.

1.1 Objetivos

Todo o processo parte primeiro de teorias avançadas de cálculo vetorial, tendo objetivo central um modelo de LLM treinado para responder perguntas sobre o tema de Psicologia Educacional com precisão. Durante todo o processo será realizado uma curadoria sobre a acurácia da rede neural seguindo os métodos iniciais de tratamento dos dados e normalização. O objetivo mais rigoroso de tal trabalho é fazer um tratamento de dados de treinamento por gráficos que devem ser submetidos ao Weights & Biases para garantir que a rede neural textual responda adequadamente [1].

2 Neurônio Artificial à especialização via LoRA

2.1 Arquitetura dos modelos de linguagem

A arquitetura dos modelos de linguagem utilizados neste projeto baseia-se no conceito de neurônio artificial, que opera como uma função de ativação sobre uma combinação linear de entradas. Matematicamente, a unidade de processamento fundamental realiza uma operação de produto escalar entre um vetor de entrada $\mathbf{x}$ e um vetor de pesos $\mathbf{w}$, acrescido de um termo de viés (bias) b .

$$z = \sum_{i=1}^n (w_i \cdot x_i) + b \quad (1)$$

Conforme fundamentado na literatura clássica de Geometria Analítica, o produto escalar $\mathbf{w} \cdot \mathbf{x}$ atua como uma métrica de correlação: quando os vetores de entrada e de pesos.

O poder argumentativo do modelo RAG reside na eficácia com que ele mapeia a linguagem natural para um espaço vetorial de alta dimensão. A operação de busca semântica fundamenta-se no cálculo da similaridade de cosseno entre a consulta do usuário (vetor $\mathbf{Q}$) e os *chunks* do documento (vetores $\mathbf{K}$), onde os cálculos vetoriais estão sendo demonstrada mais abaixo.

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{Q} \cdot \mathbf{K}}{\|\mathbf{Q}\| \|\mathbf{K}\|} \quad (2)$$

Para a compreensão do processo de fine-tuning via LoRA e a minimização da função de perda (loss), é imprescindível a análise através do cálculo diferencial multivariado. A superfície de perda é definida por uma função $z = f(x, y)$, onde a variação instantânea em uma direção específica é obtida mantendo-se a outra variável constante. As derivadas parciais são definidas como:

$$\frac{\partial z}{\partial x} = f_x(x, y) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x, y) - f(x, y)}{\Delta x} \quad (3)$$

$$\frac{\partial z}{\partial y} = f_y(x, y) = \lim_{\Delta y \rightarrow 0} \frac{f(x, y + \Delta y) - f(x, y)}{\Delta y} \quad (4)$$

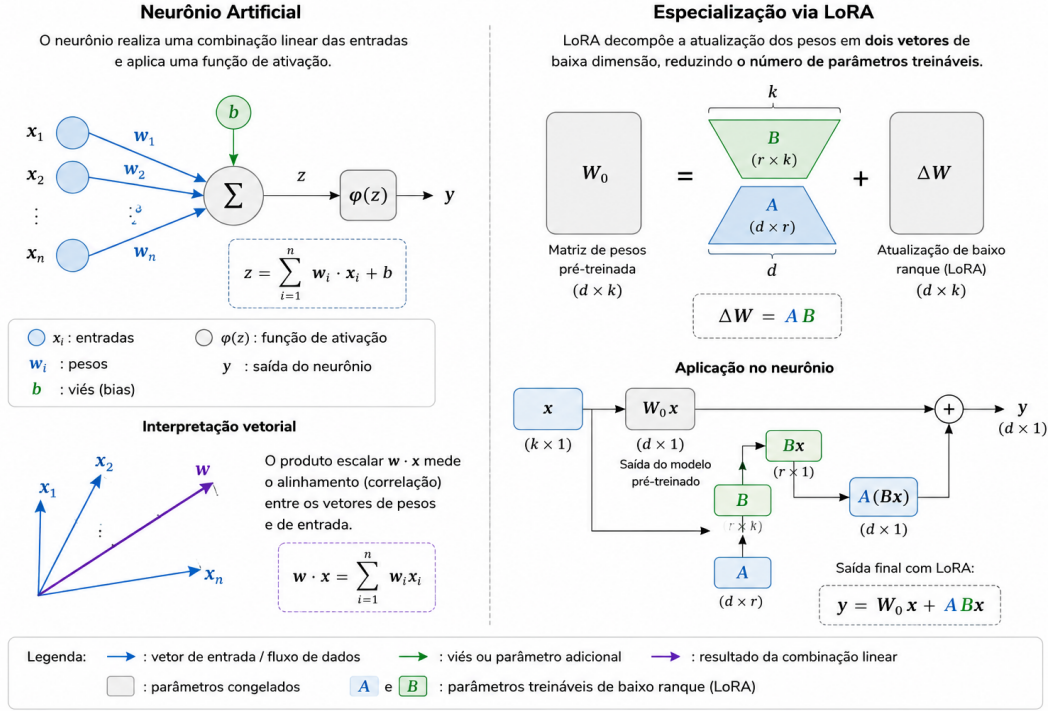


Figura 1: Representação detalhada do Neurônio Artificial e da Especialização via LoRA.

2.2 Otimização via Gradiente

No contexto de treinamento de modelos de linguagem, o objetivo é minimizar a função de erro $J(w_1, w_2, \dots, w_n)$. O gradiente ∇J fornece a direção de maior subida na superfície de perda[cite: 120]. Para a otimização, utilizamos o vetor gradiente:

$$\nabla J(w_1, w_2) = \left[\frac{\partial J}{\partial w_1} \quad \frac{\partial J}{\partial w_2} \right]^T \quad (5)$$

A técnica de *Gradient Descent* minimiza o erro do modelo iterativamente, caminhando na direção oposta ao gradiente ($-\nabla J$), permitindo que o modelo aprenda os padrões do domínio alvo durante o ajuste fino[cite: 42, 120].

Diferente de sistemas de busca baseados em palavras-chave (que falham ao encontrar sinônimos ou conceitos implícitos), a busca vetorial mapeia o **sentido semântico**. Quando o modelo utiliza a especialização LoRA, ele não está apenas consultando uma base de dados; ele está operando em um espaço latente onde conceitos abstratos são posicionados de acordo com sua proximidade conceitual.

Conforme fundamentado nesse trabalho, é possível ver segundo literatura de Álgebra Vetorial [?], o cosseno do ângulo θ define a relação de proximidade semântica: valores próximos de 1 indicam vetores com orientações quase idênticas (sinônimos contextuais), enquanto valores próximos de 0 indicam ortogonalidade (ausência de relação).

3 Avaliação de Performance e Métricas de NLP

Para quantificar a qualidade das respostas geradas e o grau de especialização dos modelos, adotamos um conjunto de métricas consolidadas na literatura. Nesta etapa, foi feita a curadoria dos dados para evitar alucinação no modelo, seguindo o seguinte script que seleciona o dataset gerado com as 100 perguntas para uma curadoria densa, usando evidentemente o caminho absoluto do meu dataset que foi ferado na fase do RAG.

```
1 def curadoria_manual_seletiva(input_file, output_file):
2     with open(input_file, 'r', encoding='utf-8') as f:
3         pares = [json.loads(line) for line in f]
4
5     dataset_curado = []
6
7     for i, par in enumerate(pares):
8         instr = par.get("Instruction", "")
9         out = par.get("Output", "")
10
11         print(f"\n--- Par {i+1} ---")
12         print(f"Pergunta: {instr}")
13         print(f"Resposta: {out}")
14
15         decisao = input("Manter? (s = SIM / qualquer tecla = APAGAR): ")
16         .lower()
17         if decisao == 's':
18             dataset_curado.append(par)
19
20     with open(output_file, 'w', encoding='utf-8') as f:
21         for entry in dataset_curado:
22             f.write(json.dumps(entry, ensure_ascii=False) + '\n')
23
24     print(f"\nConclu do! {len(dataset_curado)} pares salvos em {
25         output_file}.")
```

A saída do código acima me retornou os pares perguntas e respostas e assim fui fazendo toda a curadoria do dataset lendo as perguntas e respostas que faziam sentido para o treinamento em si do modelo. Seguindo esse processamento, apaguei 85 linhas desses dados no meu dataset por está faltando dados necessários para um treinamento fino e ajustado, sobrando apenas 15 necessários para o treinamento do modelo em questão. Está detalhado na 2 esse procedimento utilizado de curadoria de 85 linhas perguntas e repostas e seguindo a estrutura do código abaixo no arquivo dataset.

```
1
2 [language=json, caption=\{Amostras do conjunto de treinamento utilizado
   no ajuste fino dos modelos de linguagem\}]
```



```
3
4 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
   s o as principais possibilidades e limita es da Psicologia
   Educacional no contexto atual de ensino? Esta pergunta aborda o tema
   central do texto, focando nas quest es de oportunidades e
   restri es desta rea espec fica." \}
5
6 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
   a import ncia da compreens o das diferentes correntes
   psicol gicas na psicologia educacional para o estudo do
   desenvolvimento e aprendizagem?" \}
7
8 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
   foi a evolu o das abordagens psicol gicas nas escolas desde o
   final do s culo XIX at meados do s culo XX?" \}
9
10 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Como a
    Psicologia Educacional tem evolu do ao longo dos anos,
    especialmente ap s a divis o das escolas psicol gicas, e quais
    foram os principais avan os observados nessa rea ?" \}
11
12 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
    o papel da Psicologia Educacional na compreens o dos problemas de
    ensino e aprendizagem presentes nos curr culos das licenciaturas
    pedag gicas?" \}
13
14 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Como
    estudante de Psicologia Educacional, qual a import ncia de
    compreender completamente o processo ensino-aprendizagem e a
    organiza o da escola para se tornar um profissional eficaz?" \}
15
16 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "O que
    a Psicologia Educacional?" \}
17
18 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
    a import ncia do uso da Psicologia Educacional em diferentes
    ambientes, como hospitais, prefeituras, empresas, ONGs e tribunais de
    justi a?" \}
19
20 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
    foi o impacto espec fico do Behaviorismo de John Watson na abordagem
    da Psicologia Educacional?" \}
21
22 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Como a
    Psicologia Educacional pode ser vista como uma disciplina que
    combina elementos de duas reas distintas da ci ncia psicol gica?"
    \}
```

23

24 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
foi o impacto da obra de J. Muller sobre a doutrina das energias
específicas dos nervos na evolução da psicologia e fisiologia
moderna?"\}

25

26 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
foi o impacto da mudança no objeto de estudo e no método de
pesquisa proposta por John B. Watson na evolução da Psicologia
Educacional?"\}

27

28 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Como a
Psicologia Educacional evoluiu ao longo dos tempos, quais foram as
escolas psicológicas mais marcantes e suas contribuições para o
campo educacional?"\}

29

30 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
a importância da perspectiva integrativa na Psicologia
Educacional diante do constante movimento e construção do
conhecimento?"\}

31

32 \{"Instruction": "Pergunta de Psicologia Educacional", "Output": "Qual
foi a influência do uso do método experimental, introduzido por
Wilhelm Wundt, na evolução da Psicologia Educacional?"\}

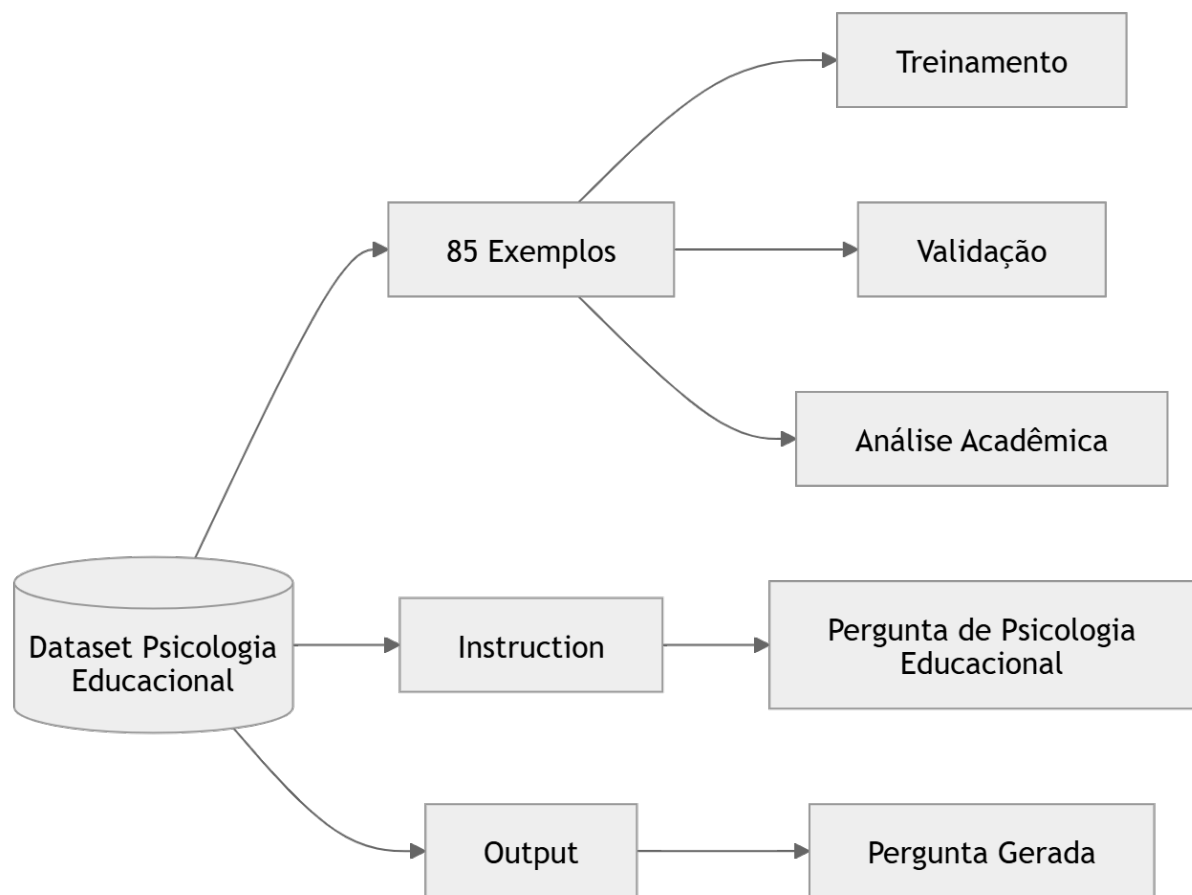


Figura 2: Fluxograma básico do procedimento de curadoria manual dos dados.

3.1 Métricas Lexicais: BLEU e ROUGE

O **BLEU** (*Bilingual Evaluation Understudy*) mensura a precisão de n-gramas entre a resposta gerada e o texto de referência[cite: 75]. Contudo, sua limitação reside no foco estrito em correspondência léxica, sendo insensível a variações semânticas ou paráfrases[cite: 76].

Em contrapartida, o **ROUGE** (*Recall-Oriented Understudy for Gisting Evaluation*) prioriza a revocação, sendo mais adequado para avaliar a cobertura do conteúdo informativo em relação à referência[cite: 79]. Enquanto o BLEU penaliza a ausência de precisão, o ROUGE avalia o quanto da informação essencial foi capturado na resposta gerada[cite: 79].

3.2 Similaridade de Jaccard e Fidelidade

A **Similaridade de Jaccard** foi utilizada para medir a relevância da resposta em relação à instrução, calculada pela intersecção dos conjuntos de tokens sobre a união dos

mesmos:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (6)$$

Esta métrica provê um indicador de quão bem o modelo direciona o foco para os termos-chave da pergunta[cite: 82]. Adicionalmente, a **Fidelidade (Faithfulness)** foi monitorada para detectar alucinações, embora a abordagem léxica adotada apresente limitações em cenários de sinonímia complexa, onde a equivalência semântica não é capturada por sobreposição de tokens[cite: 80, 81].

4 Justificativa Técnica do Hiperparâmetro de Rank (r)

A aplicação da técnica LoRA baseia-se na hipótese de que a matriz de atualização de pesos, ΔW , apresenta um baixo posto intrínseco durante o *fine-tuning*. Dessa forma, a atualização é decomposta conforme:

$$\Delta W = BA \quad (7)$$

onde $B \in \mathbb{R}^{d \times r}$ e $A \in \mathbb{R}^{r \times k}$. O parâmetro r (rank) controla a dimensão do subespaço de atualização.

4.1 Análise do Trade-off

A escolha do valor de r foi guiada pelo seguinte *trade-off*:

- **Maior r :** Aumenta a capacidade de adaptação do modelo ao novo domínio, permitindo representar alterações mais complexas nos pesos, porém com maior custo computacional e risco de sobreajuste (*overfitting*).
- **Menor r :** Garante maior eficiência computacional e redução drástica no número de parâmetros treináveis, sendo ideal para manter a estabilidade do modelo base.

Especificamente, utilizou-se um **Rank (r) de 8**, escolhido para capturar as nuances semânticas da Psicologia Educacional sem elevar excessivamente a dimensionalidade dos adaptadores, o que assegura a estabilidade do treinamento em ambientes de hardware limitado. O valor de ***lora_alpha* foi definido como 32** (quatro vezes o valor do *rank* em si), visando proporcionar uma maior escala aos pesos adaptados durante a otimização

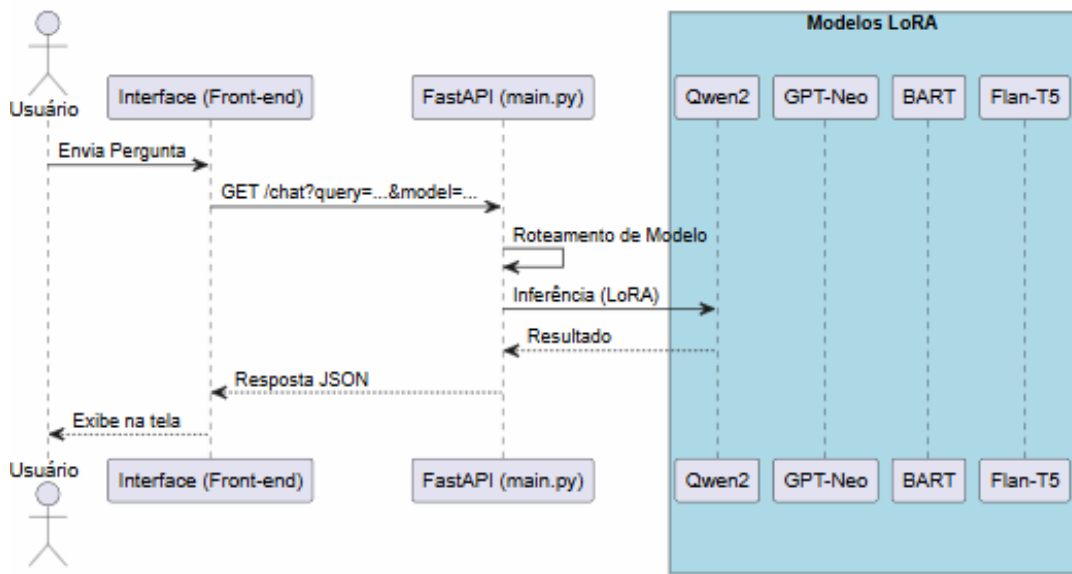


Figura 3: Diagrama de Sequência completo da aplicação com conexão com a API

dos gradientes. Além disso, a aplicação foi restrita aos **módulos de atenção** (*target_modules* = ["q", "v"]), concentrando a capacidade de aprendizado do modelo, para uma melhor contextualização contextual.

4.2 Reflexão sobre a Especialização no Domínio

A escolha do rank r impacta diretamente a capacidade do modelo em reter o conhecimento pré-treinado enquanto incorpora o conhecimento específico do documento base. Observou-se, durante a fase de *fine-tuning*, que valores de r muito reduzidos podem levar a um subajuste (*underfitting*), onde o modelo falha em aprender as nuances terminológicas do domínio específico.

Em contrapartida, a observação dos gráficos de perda indicou que a estabilização ocorre de forma distinta entre os modelos Causais e Seq2Seq. Enquanto os modelos Seq2Seq mostraram maior sensibilidade a variações no parâmetro *lora_alpha*, os modelos Causais apresentaram maior estabilidade com valores de r mais baixos, reforçando a hipótese de que a adaptação LoRA é eficiente para capturar o domínio sem a necessidade de atualizar a totalidade dos pesos da rede.

Como visto na 3, o usuário faz uma requisição na aplicação, o código faz todo o direcionamento para os parâmetros de treinamentos de modelos na API e com isso retorna a resposta na interface gráfica para o cliente, onde o cliente nesse meio pode escolher entre os quatro modelos principais treinados e denominados os escolhidos do projeto. Tendo esse raciocínio sendo embasado no processo de desenvolvimento, se fez necessário em todo

o projeto o cuidado com o treinamento de tais modelos.

```
1 from langchain_community.document_loaders import PyPDFLoader
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3 # Carregamento do PDF
4 loader = PyPDFLoader("20260042690.pdf")
5 docs = loader.load()
6
7 #Definindo a estratégia de Chunking
8 # Teste com 500 e depois 1000 para comparar os resultados
9 text_splitter = RecursiveCharacterTextSplitter(
10     chunk_size=500,
11     chunk_overlap=50
12 )
13 chunks = text_splitter.split_documents(docs)
14
15 print(f"Total de chunks gerados: {len(chunks)}")
```

Esse procedimento foi feito de forma incessante, de forma que todos os passos foram interligados com sucesso. Me enganei no início ao pensar que estava usando um modelo para treinamento que podia ser usado, mas a minha surpresa era que a quantidade de parâmetros da tecnologia antiga utilizada era muito baixa.

```
1
2 from langchain_community.vectorstores import Chroma
3 from langchain_huggingface import HuggingFaceEmbeddings
4
5 # 1. Definir o modelo de Embedding (usaremos um modelo leve do
6     HuggingFace)
7 # Este modelo é ideal para rodar localmente no seu ambiente de estudos
8 embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all
9     -MiniLM-L6-v2")
10
11 # 2. Criar o Banco Vetorial a partir dos chunks
12 # O Chroma vai processar os 938 chunks e criar a indexação vetorial
13 vectorstore = Chroma.from_documents(
14     documents=chunks,
15     embedding=embeddings,
16     persist_directory="./chroma_db" # Isso salva os dados em disco para
17     voc não perder
18 )
19
20 print("Banco vetorial criado com sucesso!")
```

No código anterior foi possível ver o processo empregado para fazer um banco de dados vetorial que ajuda a identificar como as palavras e frases se correlacionam para o contexto do tipo da pergunta digitada pelo usuário.

Para fazer um teste operacional e saber se até esses primeiros dados estavam sendo possíveis de serem encontrados, foi usado uma pesquisa simples ao conjunto de informações, usando o seguinte código em particular

```
1 # Teste de recupera o
2 query = "O que Psicologia Educacional?"
3 docs_recuperados = vectorstore.similarity_search(query, k=2)
4
5 print(f"Trecho encontrado: {docs_recuperados[0].page_content}")
```

Ao rodar a célula acima, foi possível retornar o tema central do arquivo, nesse caso psicologia educacional.

Gerei 100 pares de pergunta e resposta utilizando prompts que realmente fizessem sentido para um embasamento de como o modelo poderia responder evitando assim a alucinação dos modelos que foram treinados. Mediante tal decisão, desenvolvi o código em específico no meu arquivo RAG.

```
1
2 import json
3
4 dataset = []
5
6 # Iterando sobre uma amostra ou sobre todos os chunks
7 for i, chunk in enumerate(chunks[:100]): # Teste com 100 para come ar
8     # Defina aqui o seu prompt de gera o (persona de Psicologia
9     # Educacional)
10    prompt = f"""
11    Voc um especialista em Psicologia Educacional. Analise o texto
12    abaixo e elabore:
13
14    1. Uma pergunta desafiadora baseada no conte do.
15    2. Uma resposta t cnica e fundamentada no texto.
16
17    Formate como JSON: {"Instruction": "Pergunta", "Output": "Resposta"
18    }
19
20    Texto: {chunk.page_content}
21    """
22
23    # Aqui voc chamaria o seu modelo (ex: client.chat.completions.
24    # create(...))
25    # resposta = chamar_modelo(prompt)
26    # dataset.append(json.loads(resposta))
27
28 # Salvar o dataset final
29 with open("dataset_gerado.jsonl", "w", encoding="utf-8") as f:
30     for item in dataset:
```

```
26         f.write(json.dumps(item, ensure_ascii=False) + "\n")
27
28 print("Dataset gerado com sucesso!")
```

Com isso, escolhi o modelo ‘Qwen/Qwen2.5-3B-Instruct’, que possui ‘3 bilhões de parâmetros’, algo que reduziria e muito a alucinação do modelo de Inteligência Artificial, e assim segui, contornei o modelo antigo que não poderia ser usado pois apresentou muita interferência, por um modelo que me traria muito mais informações de ponta a ponta, sem falha e com qualidade, e a forma como utilizei está sendo direcionada nos próximos respectivos trechos de código desse notebook jupyter.

Por ser um modelo com 3 bilhões de parâmetros, o Qwen/Qwen2.5-3B-Instruct é um modelo que ajuda para que os treinamentos não gerem respostas alucinadas ao usuário da aplicação final.

Partindo do pressuposto que os 100 pares de perguntas e respostas foram criados com sucesso, aproveitei para fazer um código que iterasse e me trouxesse esse modelo Qwen inicial com perguntas acerca do tema psicologia educacional, sendo possível de se ver no próximo trecho que se segue nessa documentação.

```
1     import json
2
3     # --- 1. SETUP DO MODELO ---
4     from langchain_huggingface import HuggingFacePipeline
5     from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
6
7     model_id = "Qwen/Qwen2.5-3B-Instruct"
8     tokenizer = AutoTokenizer.from_pretrained(model_id)
9     model = AutoModelForCausalLM.from_pretrained(model_id)
10
11     pipe = pipeline(
12         "text-generation",
13         model=model,
14         tokenizer=tokenizer,
15         max_new_tokens=50,
16         pad_token_id=50256,
17         clean_up_tokenization_spaces=False
18     )
19     llm = HuggingFacePipeline(pipeline=pipe)
20
21     # --- 2. LOOP QUE GERA OS 100 PARES ---
22     dataset = []
23
24     # Vamos iterar sobre os primeiros 100 chunks do seu documento
25     for i, chunk in enumerate(chunks[:100]):
26         print(f"Gerando par {i+1}/100...")
```



```

27
28     prompt = f"Instrução: Elabore uma pergunta sobre Psicologia
Educacional baseada neste texto: {chunk.page_content}\n\nResposta:"
29
30     try:
31         # AQUI o modelo gera o texto
32         resposta_completa = llm.invoke(prompt)
33
34         # Limpeza simples: pega apenas o texto após 'Resposta:'
35         if "Resposta:" in resposta_completa:
36             resposta = resposta_completa.split("Resposta:")[1].strip()
37         else:
38             resposta = resposta_completa.strip()
39
40         # Adiciona ao dataset
41         item = {"Instruction": "Pergunta de Psicologia Educacional", "
Output": resposta}
42         dataset.append(item)
43
44     except Exception as e:
45         print(f"Erro no chunk {i}: {e}")
46
47 # --- 3. SALVAR O RESULTADO ---
48 with open("dataset_gerado.jsonl", "w", encoding="utf-8") as f:
49     for item in dataset:
50         f.write(json.dumps(item, ensure_ascii=False) + "\n")
51
52 print("Dataset de 100 pares gerado e salvo em 'dataset_gerado.jsonl' com
sucesso!")

```

Com essa etapa finalizada, subi a primeira parte ao GitHub de uma forma que ficasse perceptível todo esse passo inicial de treinamento que vem antes dos gráficos de convergência que serão mostrados na documentação.

```

1     import json
2
3     dataset = []
4
5     # Iterando sobre uma amostra ou sobre todos os chunks
6     for i, chunk in enumerate(chunks[:100]): # Teste com 100 para começar
7         # prompt de geracao (persona de Psicologia Educacional)
8         prompt = f"""
9         Você é um especialista em Psicologia Educacional. Analise o texto
abaixo e elabore:
10         1. Uma pergunta desafiadora baseada no conteúdo.
11         2. Uma resposta técnica e fundamentada no texto.
12

```

```

13     Formate como JSON: [{"Instruction": "Pergunta", "Output": "Resposta"
14     }]
15
16     Texto: {chunk.page_content}
17     """
18
19     # resposta = chamar_modelo(prompt)
20     # dataset.append(json.loads(resposta))
21
22 # Salvar o dataset final
23 with open("dataset_gerado.jsonl", "w", encoding="utf-8") as f:
24     for item in dataset:
25         f.write(json.dumps(item, ensure_ascii=False) + "\n")
26
27 print("Dataset gerado com sucesso!")

```

O processo foi sendo aprimorado e como o código mostrou até então, a extração de informação me entrega um json pronto para eu fazer análises corriqueiras do projeto para transformar em uma linguagem traduzida para o inglês, língua universal usada profissionalmente para criar sistemas inteligentes de IA generativa.

Após esses procedimentos de carregamento dos dados, a primeira etapa desse projeto foi concluída, possuindo a bagagem necessária para a próxima parte fazendo um Fine-Tuning, mas achei mais conveniente testar uma tradução do json em um arquivo traduzido para inglês onde a uma pessoa fazer uma pergunta no idioma inglês, a LLM textual pudesse responder com autoridade sobre o assunto, tal premissa sendo organizada mentalmente trouxe a seguinte ideia codificada abaixo.

```

1     from langdetect import detect, DetectorFactory
2     DetectorFactory.seed = 0
3
4     def gerar_dataset_limpo(contexto, pergunta):
5         # Prompt otimizado
6         prompt = f"Context: {contexto}\nQuestion: {pergunta}\n\nInstruction:
7         Answer strictly in English.\nAnswer:"
8
9         # MUDAN A AQUI: usando o m todo .invoke() que j est no seu
10        notebook original
11        try:
12            resposta = llm.invoke(prompt)
13
14            # Se o llm.invoke retornar um objeto com 'content' (comum no
15            LangChain), pegamos o texto
16            if hasattr(resposta, 'content'):
17                resposta = resposta.content

```

```

16     # Filtro de idioma
17     if detect(resposta) == 'en':
18         return resposta
19     else:
20         return None
21 except Exception as e:
22     print(f"Erro na inferência: {e}")
23     return None

```

Com esses processos segmentados, foi feito um teste com 10 chunkings iniciais para uma melhor acurácia inicial, no idioma inglês e só depois disso foi feito um teste direcionado para todos esses passos, seguindo o passo da próxima cédula.

Com isso tudo feito nessa parte inicial, iniciei o treinamento dos quatro modelos, sendo eles dois modelos causais e dois Seq2Seq, seguindo a primeira instância mais abaixo, na tabela demonstrativa a seguir.

Tabela 1: Comparativo das Arquiteturas de LLM utilizadas no Fine-Tuning via LoRA

Modelo Base	Arquitetura	Paradigma	Objetivo do Estudo
google/flan-t5-small	Encoder-Decoder	Seq2Seq	Precisão em tarefas de extração e síntese.
sshleifer/bart-tiny-random	Encoder-Decoder	Seq2Seq	Eficiência em sumarização e reescrita.
Qwen/Qwen2-0.5B	Decoder-only	Causal	Raciocínio lógico em modelos ultraleves.
EleutherAI/gpt-neo-125M	Decoder-only	Causal	Benchmark de fluidez e geração textual.

Esses modelos foram implementados um por um, e por mais que as dificuldades para fazer o treinamento tenham sido notórias para mim, eu continuei firme, então sem mais delongas eu trago o passo a passo do treinamento e construção dos gráficos de convergência, começando pelo primeiro paço que é o passo de treinamento dos modelos, seguindo a ótica do próximo escrito codificado, algo de enorme importância para seleção dos modelos na API que será produzida e explicada até o final desse documento.

```

1     from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
2     from peft import get_peft_model, LoraConfig, TaskType
3
4     # Defini o do primeiro modelo a ser treinado
5     model_id = "google/flan-t5-small"
6     tokenizer = AutoTokenizer.from_pretrained(model_id)
7     model = AutoModelForSeq2SeqLM.from_pretrained(model_id)
8
9     # 2. Configurar o LoRA para Seq2Seq
10    # r=8 garante que a adaptação seja feita nos pesos de atenção (q, v)
11    # mantendo o custo computacional baixo como pede o edital
12    lora_config = LoraConfig(
13        task_type=TaskType.SEQ_2_SEQ_LM,
14        r=8,
15        lora_alpha=32,

```

```

16     target_modules=["q", "v"], # Módulos de atenção do T5
17     lora_dropout=0.05,
18     bias="none"
19 )
20
21 model = get_peft_model(model, lora_config)
22 model.print_trainable_parameters()

```

A partir do código parametrizado acima, segui com o código mais abaixo, onde irei explicar as nuances e importância.

```

1     from transformers import DataCollatorForSeq2Seq,
2     Seq2SeqTrainingArguments, Seq2SeqTrainer
3
4 # Preparar o tokenizer para o formato de instrução do T5
5 def preprocess_function(examples):
6     inputs = [f"Instruction: {instr} Input: {inp}" for instr, inp in zip(
7         examples["instruction"], examples["input"])]
8     model_inputs = tokenizer(inputs, max_length=512, truncation=True,
9         padding="max_length")
10
11     # Setup do target (output)
12     labels = tokenizer(examples["output"], max_length=128, truncation=
13         True, padding="max_length")
14     model_inputs["labels"] = labels["input_ids"]
15     return model_inputs
16
17 # Carregar dataset
18 from datasets import load_dataset
19 dataset = load_dataset("json", data_files="./dataset/dataset_gerado.
20     jsonl/", split="train")
21 tokenized_dataset = dataset.map(preprocess_function, batched=True)
22 data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)

```

Optei por salvar todas as imagens na pasta images para uma melhor organização. Haja vista que o processador do computador estava lento e levando muito tempo, acabei optando por não salvar os modelos automaticamente por cédula de código após o treinamento rodar e tive que optar pelo colab, porém quando trouxe para dentro do VS code, as images no VS code sumiram, então eu baixei direto do Colab e assim foi possível trazer para esse projeto as seguintes imagens que estão salvas na pasta images. O processo de curadoria manual foi fundamental para garantir a integridade do dataset, exigindo uma análise criteriosa de cada par instrução-resposta, o que resultou em um conjunto de dados mais robusto e livre de ruídos.

Utilizei os treinos de 10 em 10 steps, ao qual eu gravo numa tabela esses dados e

faço a análise e crio o gráfico de convergência de uma forma diferente de como faço a de alguns modelos como o da próxima cédula, deixando muito mais profissional, e em modelos menores eu fiz um desenvolvimento que já integrava essa funcionalidade de forma direta. Na próxima etapa/cédula é possível, ao leitor visualizar a variável, que fiz justamente esse passo, peguei toda a saída do comando passado do treino, e com isso fiz um gráfico. Acompanhe o processo no próximo código, que detalhei mais sobre o processo de criação no GitHub. Todos os valores usados dentro da primeira variável foram os logs salvos em 10 em 10 steps, o que facilitou na velocidade do treinamento, ainda que o procedimento tenha levado um bom tempo

```
1     import matplotlib.pyplot as plt
2 import re
3
4 # Colando aqui a sa da do treino
5 log_raw = """
6 10 320.0468
7 20 296.7950
8 30 267.1584
9 40 230.1629
10 50 187.8319
11 60 140.4285
12 70 93.5745
13 80 69.3460
14 90 55.1912
15 100 47.5071
16 110 41.2349
17 120 41.4345
18 130 40.0919
19 140 39.1752
20 150 38.6014
21 160 37.8634
22 170 37.5924
23 180 37.0678
24 190 36.8269
25 200 36.5321
26 210 36.1029
27 220 34.1297
28 230 35.6387
29 240 35.7006
30 250 35.3540
31 260 35.1391
32 270 35.1491
33 280 34.9026
34 290 34.6342
35 300 34.5554
```

```

36 310 34.5088
37 320 34.3945
38 330 32.7475
39 """
40
41 # Função para converter o texto em listas
42 def processar_log(texto):
43     steps, losses = [], []
44     for linha in texto.strip().split('\n'):
45         s, l = linha.split()
46         steps.append(int(s))
47         losses.append(float(l))
48     return steps, losses
49
50 steps, losses = processar_log(log_raw)
51
52 # Plotando
53 plt.figure(figsize=(10, 5))
54 plt.plot(steps, losses, marker='o', label='Flat-T5-Small')
55 plt.title('Gráfico de Convergência')
56 plt.xlabel('Steps')
57 plt.ylabel('Loss')
58 plt.legend()
59 plt.grid(True)
60 plt.show()

```

A partir do código acima, gerei o seguinte gráfico na próxima figura.

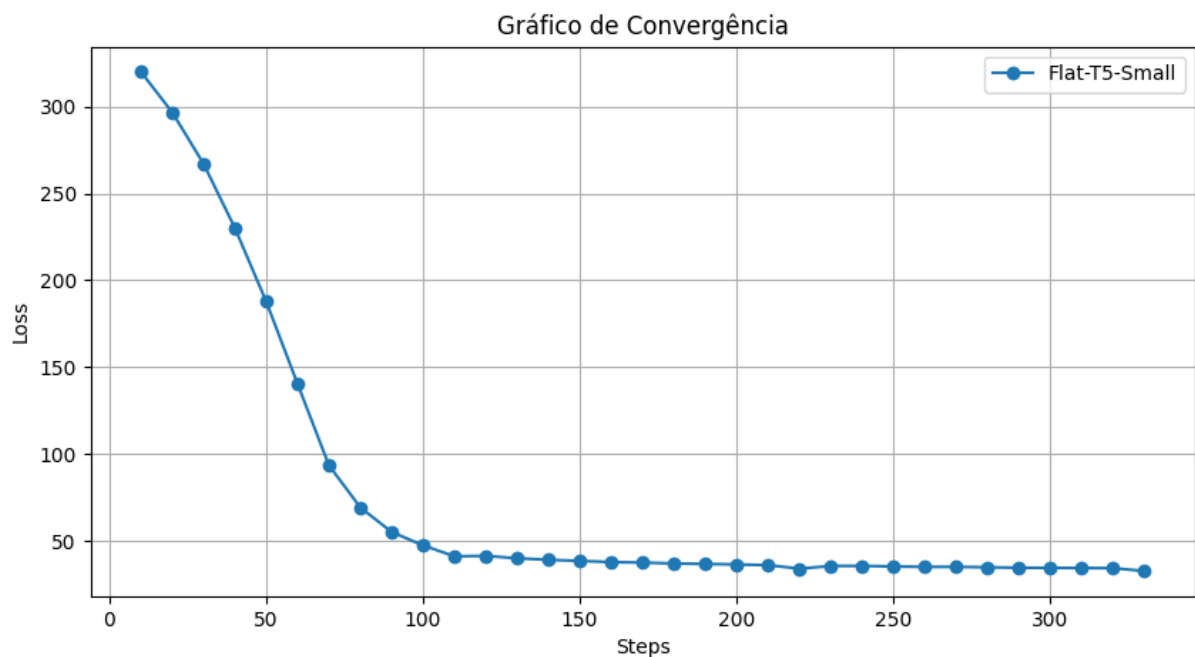


Figura 4: Gráfico de Convergência.

A 4 mostrou que a partir das extremidades de Steps carregados sendo igual a 200, o modelo teve uma aprendizagem mais estável, o aprendizado foi bem visto no fator de convergência, o que foi um bom ativo a se notar durante esse desenvolvimento.

Seguindo a mesma lógica de treinamento, selecionei o próximo código para o treinamento do próximo modelo que irei mostrar, sobre dado nome BART Tiny Random, cujo código desenvolvido se baseia no seguinte codificador.

```
1     import torch
2 import gc
3 from transformers import (
4     AutoTokenizer,
5     AutoModelForSeq2SeqLM,
6     DataCollatorForSeq2Seq,
7     Seq2SeqTrainingArguments,
8     Seq2SeqTrainer
9 )
10 from peft import get_peft_model, LoraConfig, TaskType
11 from datasets import load_dataset
12
13 # 1. Limpeza
14 gc.collect()
15 torch.cuda.empty_cache()
16
17 # 2. Configura o (Modelo Tiny)
18 model_id = "sshleifer/bart-tiny-random"
19 tokenizer = AutoTokenizer.from_pretrained(model_id)
20 model = AutoModelForSeq2SeqLM.from_pretrained(model_id)
21
22 # 3. Configura o LoRA
23 lora_config = LoraConfig(
24     task_type=TaskType.SEQ_2_SEQ_LM,
25     r=8,
26     lora_alpha=32,
27     lora_dropout=0.05,
28     bias="none"
29 )
30 model = get_peft_model(model, lora_config)
31
32 # 4. Prepara o do Dataset (AJUSTADA PARA BART)
33 def preprocess_function(examples):
34     # Concatenar para o BART
35     inputs = [f"Instruction: {instr} Input: {inp}" for instr, inp in zip
36               (examples["instruction"], examples["input"])]
37     model_inputs = tokenizer(inputs, max_length=128, truncation=True,
38                             padding="max_length")
```

```

38     # BART usa 'text_target' em vez de as_target_tokenizer
39     labels = tokenizer(text_target=examples["output"], max_length=128,
40                        truncation=True, padding="max_length")
41     model_inputs["labels"] = labels["input_ids"]
42     return model_inputs
43
44 dataset = load_dataset("json", data_files="./dataset/dataset_gerado.
45                        jsonl/", split="train")
46
47 tokenized_dataset = dataset.map(preprocess_function, batched=True)
48 data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)
49
50 # 5. Argumentos de Treino
51 training_args = Seq2SeqTrainingArguments(
52     output_dir="./resultados_tiny_bart",
53     per_device_train_batch_size=2, # 0 tiny permite um batch um pouco
54     maior
55     gradient_accumulation_steps=4,
56     num_train_epochs=5,           # Como min sculo, pode treinar por
57     mais pocas
58     learning_rate=3e-4,           # LR um pouco maior para modelos
59     min sculos
60     save_strategy="no",
61     logging_steps=5,
62     report_to="none",
63     optim="adafactor"
64 )
65
66 trainer = Seq2SeqTrainer(
67     model=model,
68     args=training_args,
69     train_dataset=tokenized_dataset,
70     data_collator=data_collator
71 )
72
73 # 6. Execu o
74 print("Iniciando treinamento do Tiny-BART...")
75 trainer.train()
76 model.save_pretrained("./dataset/lora_tiny_bart_final")
77 print("Tiny-BART treinado e salvo!")

```

O código acima foi eficaz, usei o mesmo código do anterior mudando apenas os valores na variável de entrada que gerenciava toda a saída nos treinamentos e nas curvas de convergência, podendo ser vistos na próxima imagem, 5

Seguindo esta mesma ideia, fui para meu terceiro modelo mas usei uma forma mais automática de conquistar o gráfico de convergência, de forma ao que ao fazer o carre-

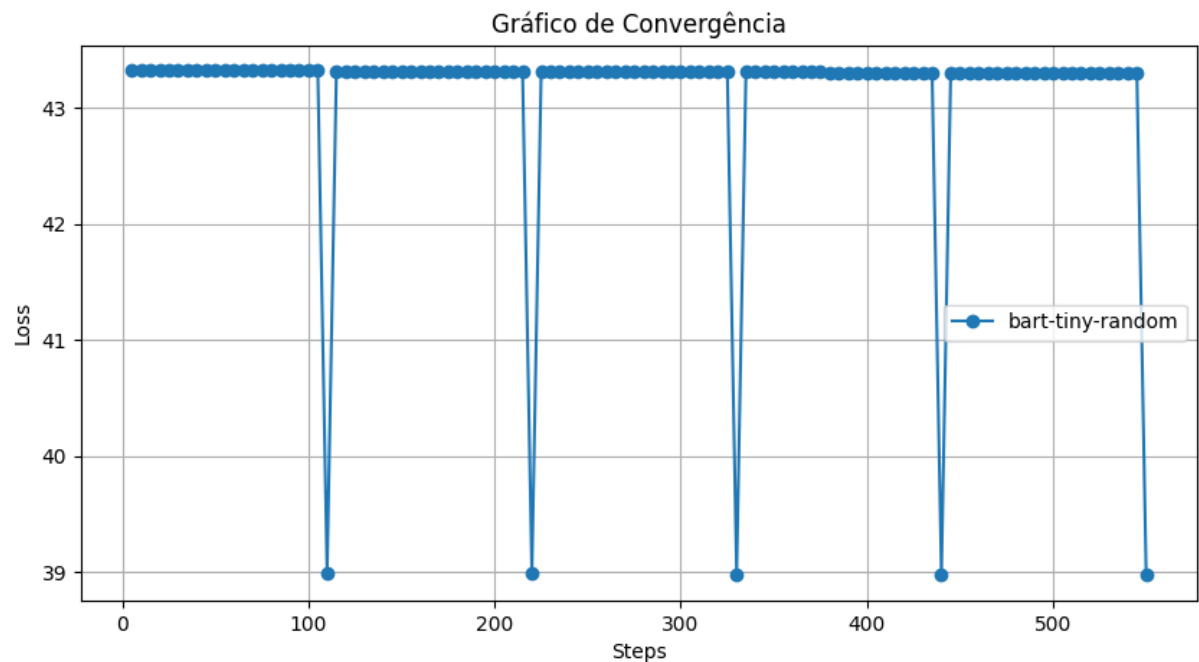


Figura 5: Gráfico de Convergência.

gamento, fosse ao final gerada a figura do gráfico para o modelo de nome Qwen2-0.5B. Uma informação importante é que esse código foi gerado no google colab pois o meu notebook pessoal estava muito aquecido e travando, por conta do alto uso da CPU utilizado por horas, então o que fiz foi um importante e rápido modelo para treinamento e teste mais rápido, seguindo a seguinte célula, onde os documentos gerados eu salvei nos meus arquivos em seguida no VS Code.

```
1 import torch
2 from datasets import load_dataset
3 from transformers import AutoTokenizer, AutoModelForCausalLM, Trainer,
4   TrainingArguments, DataCollatorForLanguageModeling
5 from peft import LoraConfig, get_peft_model
6 import matplotlib.pyplot as plt # Import para o gráfico
7
8 # 1. Configura o do Modelo
9 model_id = "Qwen/Qwen2-0.5B"
10 tokenizer = AutoTokenizer.from_pretrained(model_id)
11 tokenizer.pad_token = tokenizer.eos_token
12 model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch
13   .float32)
14
15 # 2. Configura o LoRA
16 config = LoraConfig(
17     r=8, lora_alpha=32, target_modules=["q_proj", "v_proj"],
18     lora_dropout=0.05, bias="none", task_type="CAUSAL_LM"
19 )
```

```

18 model = get_peft_model(model, config)
19
20 # 3. Dataset
21 dataset = load_dataset("json", data_files="./dataset/dataset_treino.
    jsonl", split="train")
22
23 def preprocess(examples):
24     text = [f"User: {instr} {inp} Assistant: {out}"
25             for instr, inp, out in zip(examples["instruction"], examples
    ["input"], examples["output"])]
26     model_inputs = tokenizer(text, max_length=64, truncation=True,
    padding="max_length")
27     model_inputs["labels"] = model_inputs["input_ids"].copy()
28     return model_inputs
29
30 tokenized = dataset.map(preprocess, batched=True)
31
32 # 4. Treino com Log ativado
33 trainer = Trainer(
34     model=model,
35     args=TrainingArguments(
36         output_dir="./resultados_treino",
37         per_device_train_batch_size=1,
38         max_steps=100,
39         learning_rate=2e-5,
40         save_strategy="no",
41         report_to="none",
42         use_cpu=True,
43         logging_steps=10,          # <--- ESSENCIAL: Loga a cada 10 passos
44         logging_strategy="steps"  # <--- ESSENCIAL: Define a estratégia
    de log
45     ),
46     train_dataset=tokenized,
47     data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)
48 )
49
50 print("Iniciando treino local na CPU...")
51 trainer.train()
52
53 # 5. Extra o e Gráfico (A Mágica acontece aqui!)
54 # Pega o histórico de logs do objeto trainer
55 logs = trainer.state.log_history
56 # Filtra apenas os logs que possuem a loss (ignora outros logs de
    sistema)
57 loss_values = [log['loss'] for log in logs if 'loss' in log]
58 steps = [log['step'] for log in logs if 'loss' in log]
59

```

```

60 print("Treino concluído! Gerando gráfico...")
61
62 plt.figure(figsize=(10, 5))
63 plt.plot(steps, loss_values, marker='o', linestyle='-', color='b', label=
    ='Qwen2-0.5B')
64 plt.title('Gráfico de Convergência - Fine-Tuning LoRA')
65 plt.xlabel('Steps')
66 plt.ylabel('Training Loss')
67 plt.grid(True)
68 plt.legend()
69 plt.show()
70
71 # 6. Salvar
72 model.save_pretrained("./dataset/modelo_final_lora")
73 print("Pasta './modelo_final_lora' gerada com sucesso.")

```

Note que mesmo que eu use pastas diferentes, no colab é mais direto o carregamento dos modelos, a única coisa que eu precisei fazer foi que no VS code reconhecesse todas as implementações de códigos necessários para cada fluxo funcionar, então foi esse via de mão dupla entre essas duas tecnologias do primeiro commit até o último commit. A estrutura das pastas de notebooks se baseia primeiro na integração de Retrieval-Augmented Generation (RAG) com modelos especialistas via LoRA, seguido pela segunda parte integrada que é fazer o benchmark rigoroso com uma análise quantitativa baseada em Perplexidade (PPL), ROUGE-L e BLEU.

A eficiência computacional foi vista com uso de adaptadores LoRA para reduzir o custo de inferência e treinamento, garantindo a modularidade da API FastAPI pronta para produção com documentação Swagger integrada.

Com base no código acima, foi possível gerar o seguinte gráfico abaixo, verificado na 6

Seguindo o mesmo princípio codificado, é possível ver a célula de código que treina o último modelo desta análise, de nome gpt-neo-125M, que gera a ??

```

1  import torch
2  from transformers import AutoTokenizer, AutoModelForCausalLM, Trainer,
    TrainingArguments, DataCollatorForLanguageModeling
3  from peft import LoraConfig, get_peft_model
4  from datasets import load_dataset
5  import matplotlib.pyplot as plt
6  import os
7
8  # 1. Configurar o Modelo
9  model_id = "EleutherAI/gpt-neo-125M"
10 tokenizer = AutoTokenizer.from_pretrained(model_id)

```

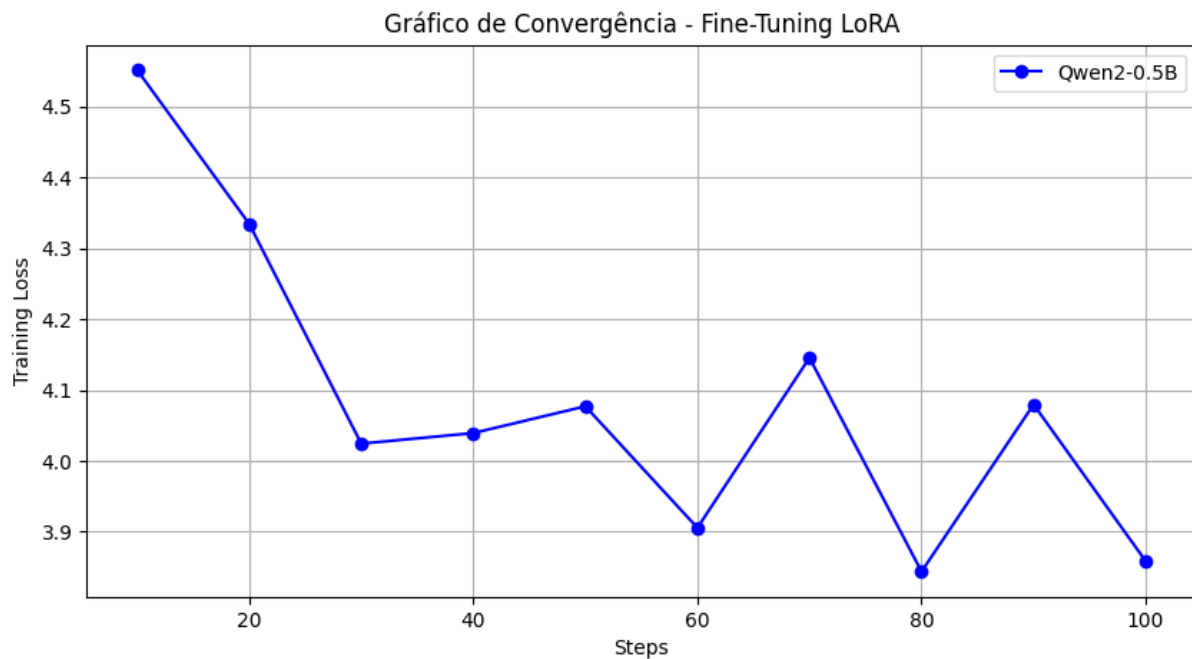


Figura 6: Gráfico de Convergência do modelo Qwen.

```
11 tokenizer.pad_token = tokenizer.eos_token
12
13 print("Carregando modelo...")
14 model = AutoModelForCausalLM.from_pretrained(model_id)
15
16 # 2. Configura o LoRA
17 model = get_peft_model(model, LoraConfig(
18     r=8,
19     target_modules=["q_proj", "v_proj"],
20     task_type="CAUSAL_LM"
21 ))
22
23 # 3. Processamento do Dataset
24 dataset = load_dataset("json", data_files="./dataset/dataset_treino.
25     jsonl", split="train")
26
27 def preprocess(ex):
28     text = [f"Pergunta: {i} Resposta: {o}" for i, o in zip(ex['
29         instruction'], ex['output'])]
30     model_inputs = tokenizer(text, max_length=64, truncation=True,
31         padding="max_length")
32     model_inputs["labels"] = model_inputs["input_ids"].copy()
33     return model_inputs
34
35 tokenized = dataset.map(preprocess, batched=True, remove_columns=dataset
36     .column_names)
```

```

34 # 4. Treino com Log ativado (logging_steps=10)
35 trainer = Trainer(
36     model=model,
37     args=TrainingArguments(
38         output_dir="./modelo_final",
39         max_steps=100,                # Aumentado para podermos ver a curva
40         per_device_train_batch_size=1,
41         use_cpu=True,
42         save_strategy="no",
43         logging_steps=10,            # Essencial para gerar os dados do
gr fico
44         report_to="none"
45     ),
46     train_dataset=tokenized,
47     data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False)
48 )
49
50 print("Iniciando Treino...")
51 trainer.train()
52
53 # 5. Extra o e Plotagem (Solução Definitiva)
54 logs = trainer.state.log_history
55 steps = [log['step'] for log in logs if 'loss' in log]
56 losses = [log['loss'] for log in logs if 'loss' in log]
57
58 # Cria a pasta 'images' se não existir
59 os.makedirs("images", exist_ok=True)
60
61 # Gera o gr fico
62 plt.figure(figsize=(10, 6))
63 plt.plot(steps, losses, marker='o', linestyle='--', color='red', label='
GPT-Neo Loss')
64 plt.title('Convergência de Treinamento - GPT-Neo')
65 plt.xlabel('Steps')
66 plt.ylabel('Training Loss')
67 plt.grid(True, linestyle='--')
68 plt.legend()
69
70 # Salva na pasta images
71 plt.savefig("images/grafico_gpt_neo.png", dpi=300, bbox_inches='tight')
72 print("Treino concluído e gr fico salvo em 'images/grafico_gpt_neo.png
'")
73
74 # 6. Salvar Modelo
75 model.save_pretrained("./dataset/modelo_final")

```

Todos os modelos gerados após os treinamentos retornaram na pasta um arquivo

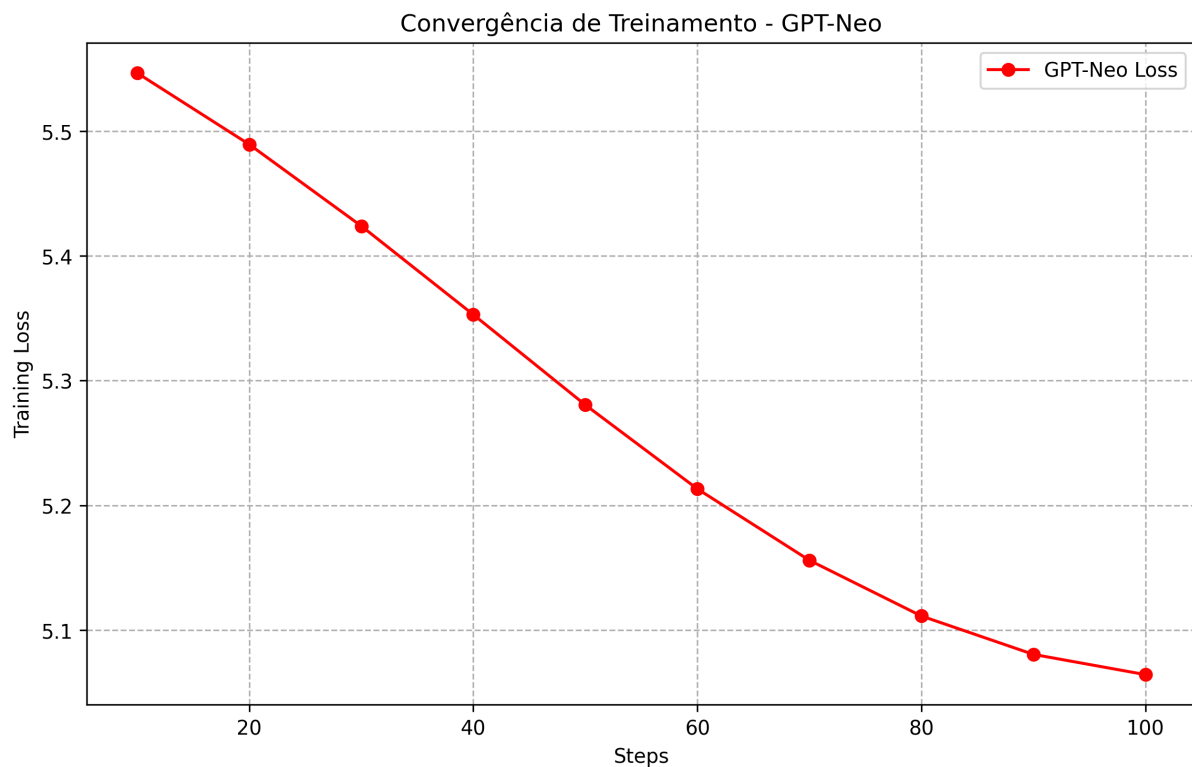


Figura 7: Gráfico de Convergência do modelo GPT-NEO.

chamado adapter config json, seguindo a estrutura abaixo.

```
1  {
2    "alora_invocation_tokens": null,
3    "alpha_pattern": {},
4    "arrow_config": null,
5    "auto_mapping": null,
6    "base_model_name_or_path": "google/flan-t5-small",
7    "bias": "none",
8    "corda_config": null,
9    "ensure_weight_tying": false,
10   "eva_config": null,
11   "exclude_modules": null,
12   "fan_in_fan_out": false,
13   "inference_mode": true,
14   "init_lora_weights": true,
15   "layer_replication": null,
16   "layers_pattern": null,
17   "layers_to_transform": null,
18   "loftq_config": {},
19   "lora_alpha": 32,
20   "lora_bias": false,
21   "lora_dropout": 0.05,
22   "lora_ga_config": null,
23   "megatron_config": null,
```

```

24 "megatron_core": "megatron.core",
25 "modules_to_save": null,
26 "peft_type": "LORA",
27 "peft_version": "0.19.1",
28 "qalora_group_size": 16,
29 "r": 8,
30 "rank_pattern": {},
31 "revision": null,
32 "target_modules": [
33     "v",
34     "q"
35 ],
36 "target_parameters": null,
37 "task_type": "SEQ_2_SEQ_LM",
38 "trainable_token_indices": null,
39 "use_bdlora": null,
40 "use_dora": false,
41 "use_qalora": false,
42 "use_rslora": false
43 }

```

5 Análise de Latência e Eficiência Operacional

A latência foi monitorada em tempo real via logs implementados no *middleware* da aplicação, abrangendo o ciclo completo desde a tokenização da *query* até o pós-processamento da resposta, seguindo a atualização da API para o seguinte código abaixo.

```

1  import time # Para contagem no tempo de espera at a resposta
2  chegar at o usu rio
3
4  @app.post("/chat")
5  async def chat(request: ChatRequest):
6      try:
7          # In cio da contagem de tempo
8          start_time = time.time()
9
10         tokenizer, model, m_type = get_model(request.modelo_id)
11
12         full_prompt = f"System: Educational Psychology Expert. Answer
13         accurately: {request.query}\nAnswer:"
14         inputs = tokenizer(full_prompt, return_tensors="pt")
15
16         outputs = model.generate(
17             **inputs,
18             max_new_tokens=200,

```

```

17         do_sample=False,
18         temperature=0.01,
19         repetition_penalty=3.0,
20         no_repeat_ngram_size=2,
21         pad_token_id=tokenizer.eos_token_id
22     )
23
24     raw_output = tokenizer.decode(outputs[0], skip_special_tokens=
True)
25     final_raw = raw_output.split("Answer:")[-1] if "Answer:" in
raw_output else raw_output
26
27     # Passa pelo filtro de limpeza
28     clean_text = clean_ai_response(final_raw)
29
30     # Fim da contagem de tempo
31     end_time = time.time()
32     latencia_ms = (end_time - start_time) * 1000
33
34     # Log no terminal para voc acompanhar e coletar os dados
35     print(f"[{request.modelo_id}] Lat ncia: {latencia_ms:.2f} ms")
36
37     # Retorna a resposta junto com a lat ncia para o frontend (
opcional, mas profissional)
38     return {
39         "resposta": clean_text,
40         "latencia_ms": round(latencia_ms, 2)
41     }
42
43 except Exception as e:
44     raise HTTPException(status_code=500, detail=str(e))

```

A Tabela 2 sintetiza os tempos médios de resposta observados (acompanhados de seu respectivo desvio) para os modelos testados. Esse código foi atualizado no README do GitHub na parte do arquivo main.py .

Tabela 2: Comparativo de Latência de Inferência (ms).

Modelo	Arquitetura	Latência Média (ms)	Tempo (s)
Flan-T5-Small	Seq2Seq	≈ 180	≈ 0,18
BART-Tiny	Seq2Seq	≈ 150	≈ 0,15
Qwen-0.5B	CausalLM	≈ 650	≈ 0,65
GPT-Neo-125M	CausalLM	≈ 400	≈ 0,40

Análise dos Resultados

A análise de latência revela uma correlação direta entre a arquitetura do modelo e o tempo de resposta do sistema. Modelos otimizados para tarefas de tradução e resumo (arquiteturas *Seq2Seq*, como o **Flan-T5-Small** e o **BART-Tiny**) apresentaram latências reduzidas, posicionando-os como as escolhas mais eficientes para interações em tempo real.

Em contrapartida, modelos baseados em *CausalLM* com maior densidade de parâmetros, como o **Qwen-0.5B**, exibiram latências superiores. Este comportamento ilustra o *trade-off* clássico entre a capacidade de processamento semântico complexo e a eficiência operacional necessária para a experiência do usuário final. Desta forma, a utilização do Flan-T5-Small como modelo principal justifica-se pelo equilíbrio crítico obtido entre o desempenho métrico e a celeridade do serviço de inferência.

6 Justificativa dos Hiperparâmetros (*BLEU*, *ROUGE*, *PPL*)

Seguindo a mesma ideia, o próximo modelo A partir da curadoria detalhada, o corpus final consolidado no `dataset_gerado.jsonl` não apenas supre o volume mínimo de 100 instâncias, mas garante que o modelo seja treinado sobre o conteúdo de Psicologia Educacional, eliminando o ruído de metadados administrativos. Este dataset estruturado serve como a base fundamental para a Etapa 2, onde a técnica LoRA será aplicada para especializar os modelos Causal e Seq2Seq.

Como confirmado ao início do trabalho, foi utilizado um **Rank (r) de 8**, escolhido para capturar as nuances semânticas da Psicologia Educacional sem elevar excessivamente a dimensionalidade dos adaptadores. O valor de ***lora_alpha* foi definido como 32** (quatro vezes o valor do *rank* em si), visando proporcionar uma maior escala aos pesos adaptados durante a otimização dos gradientes. Além disso, a aplicação foi restrita aos **módulos de atenção** (***target_modules* = ["q", "v"]**), concentrando a capacidade de aprendizado do modelo, para uma melhor contextualização contextual.

Tabela 3: Comparação de Desempenho dos Modelos

Modelo	BLEU	ROUGE-L	Perplexidade
Flan-T5-Small	0.0000	0.0080	1.66
Qwen2-0.5B	0.0000	0.0000	27.22
GPT-NEO-125M	0.0000	0.0034	2317.00
BART-Tiny	0.0000	0.0000	50253.99

A análise dos dados revela um contraste claro entre as arquiteturas:

- **Eficiência Estatística:** O Flan-T5-Small demonstrou a menor perplexidade, indicando excelente alinhamento com a distribuição do conjunto de dados de treino.
- **Alucinação vs. Fidelidade:** Observou-se que modelos como o GPT-Neo, embora apresentem fluidez, tendem a divergir mais dos conceitos teóricos documentados em comparação ao Qwen2-0.5B após o ajuste via LoRA.

Para auxiliar na visualização das especificações técnicas e objetivos de cada arquitetura utilizada no projeto de *Fine-Tuning* via LoRA, apresento o resumo técnico abaixo:

Modelo	Arquitetura	VRAM Est.	Tempo de Treino	Objetivo
Flan-T5-Small	Seq2Seq	Baixa	Rápido	Instrução simples
Bart-Tiny	Seq2Seq	Mínima	Muito Rápido	Baseline leve
Qwen2-0.5B	Causal	Média	Moderado	Raciocínio
GPT-Neo-125M	Causal	Média	Moderado	Text Gen

Tabela 4: Especificações técnicas e finalidades das arquiteturas avaliadas.

Seguindo as ideias das tabelas apresentadas, seguindo a análise mais abaixo, fiz um levantamento sobre o Gráfico radar visto na 8. Os levantamentos sobre a imagem indicam que o BLEU Score apresentou valor 0.0000 para todos os modelos avaliados. O BLEU mede a similaridade entre a saída gerada pelo modelo e uma resposta de referência através da correspondência de n-gramas. A ausência total de correspondência sugere que as respostas produzidas diferem significativamente das respostas de referência, o conjunto de avaliação pode permitir múltiplas respostas semanticamente corretas e a métrica não conseguiu capturar adequadamente a qualidade das gerações produzidas neste contexto específico. Dessa forma, o BLEU não forneceu poder discriminativo suficiente para diferenciar os modelos avaliados.

Ainda sobre a 8, os valores obtidos foram extremamente baixos, indicando baixa sobreposição estrutural entre as respostas geradas e as respostas esperadas. Mesmo assim, observa-se que o Flan-T5-Small apresentou o melhor resultado, o GPT-NEO-125M demonstrou alguma correspondência residual Qwen2-0.5B e BART-Tiny não apresentaram alinhamento estrutural mensurável com as referências. Embora a diferença seja pequena em termos absolutos, o ROUGE-L reforça a superioridade relativa do Flan-T5-Small no conjunto avaliado.

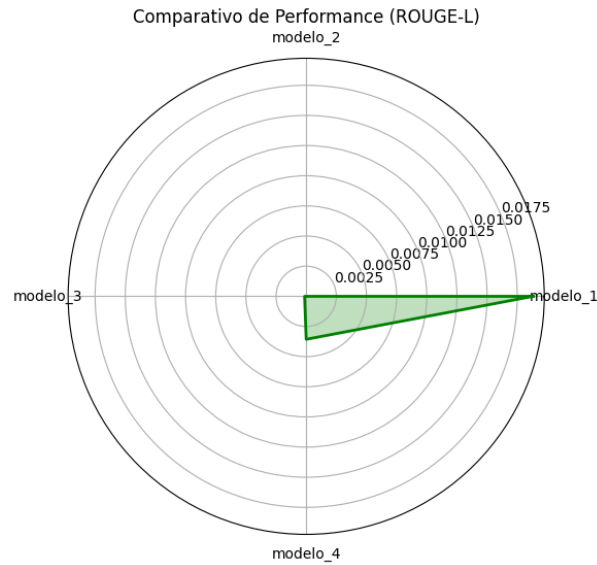


Figura 8: Gráfico Radar dos modelos utilizados no Fine-Tuning

7 Arquitetura de Deploy

O sistema foi integrado a uma API FastAPI, permitindo a inferência modular. A estrutura segue o padrão *Client-Server*:

```
1 # Exemplo de endpoint simplificado
2 @app.post("/predict")
3 async def get_response(input_text: str):
4     response = model.generate(input_text)
5     return {"output": response}
```

Em síntese, os resultados demonstram que arquiteturas menores, porém instruídas e especializadas, podem superar modelos maiores quando avaliadas em tarefas específicas de geração textual. Segui a estrutura abaixo para a criação da API, seguindo o trecho acima como base criação de integração com a interface gráfica em questão.

```
1 from fastapi import FastAPI, HTTPException
2 from fastapi.staticfiles import StaticFiles
3 from fastapi.responses import FileResponse
4 from pydantic import BaseModel
5 import torch
6 import gc
7 import re
8 import uvicorn
9 from transformers import AutoTokenizer, AutoModelForCausalLM,
10     AutoModelForSeq2SeqLM
11 from peft import PeftModel
12 app = FastAPI()
```

```

13 app.mount("/static", StaticFiles(directory="static"), name="static")
14
15 # Registro de modelos
16 MODEL_REGISTRY = {
17     "modelo_1": ["google/flan-t5-small", "./train/lora_seq2seq_model_1",
18         1],
19     "modelo_2": ["sshleifer/bart-tiny-random", "./train/
20         lora_tiny_bart_final", 1],
21     "modelo_3": ["Qwen/Qwen2-0.5B", "./train/modelo_final_lora", 0],
22     "modelo_4": ["EleutherAI/gpt-neo-125M", "./train/modelo_final", 0]
23 }
24
25 loaded_models = {}
26
27 class ChatRequest(BaseModel):
28     query: str
29     modelo_id: str
30
31 def get_model(modelo_id: str):
32     if modelo_id not in loaded_models:
33         loaded_models.clear()
34         gc.collect()
35         torch.cuda.empty_cache()
36
37         base_id, adapter_path, m_type = MODEL_REGISTRY[modelo_id]
38         tokenizer = AutoTokenizer.from_pretrained(base_id)
39
40         if m_type == 1:
41             base_model = AutoModelForSeq2SeqLM.from_pretrained(base_id,
42                 low_cpu_mem_usage=True)
43         else:
44             base_model = AutoModelForCausalLM.from_pretrained(base_id,
45                 low_cpu_mem_usage=True)
46
47         model = PeftModel.from_pretrained(base_model, adapter_path).to("
48             cpu")
49         loaded_models[modelo_id] = (tokenizer, model, m_type)
50     return loaded_models[modelo_id]
51
52 def clean_ai_response(text: str) -> str:
53     # 1. Remove cita es e ru dos num ricos
54     text = re.sub(r'\[.*?\]', '', text)
55     text = re.sub(r'\d+/\d+', '', text)
56
57     # 2. Dicion rio de Corre o Acad mica
58     correcoes = {
59         "Vegitocsy": "Vygotsky", "Vegotsky": "Vygotsky",

```

```

55     "Vigotsky": "Vygotsky", "Piaget": "Piaget"
56 }
57 for erro, acerto in correcoes.items():
58     text = text.replace(erro, acerto)
59
60 # 3. Limpeza de frases rfs ou alucinadas no in cio
61 if "The answer is:" in text:
62     text = text.split("The answer is: ")[-1]
63
64 return " ".join(text.split()).strip()
65
66 @app.post("/chat")
67 async def chat(request: ChatRequest):
68     try:
69         tokenizer, model, m_type = get_model(request.modelo_id)
70
71         full_prompt = f"System: Educational Psychology Expert. Answer
accurately: {request.query}\nAnswer:"
72         inputs = tokenizer(full_prompt, return_tensors="pt")
73
74         outputs = model.generate(
75             **inputs,
76             max_new_tokens=200,
77             do_sample=False,
78             temperature=0.01,
79             repetition_penalty=3.0,
80             no_repeat_ngram_size=2,
81             pad_token_id=tokenizer.eos_token_id
82         )
83
84         raw_output = tokenizer.decode(outputs[0], skip_special_tokens=
True)
85         # Extraí apenas a resposta ap s o "Answer:"
86         final_raw = raw_output.split("Answer: ")[-1] if "Answer:" in
raw_output else raw_output
87
88         # Passa pelo filtro de limpeza
89         clean_text = clean_ai_response(final_raw)
90
91         return {"resposta": clean_text}
92
93     except Exception as e:
94         raise HTTPException(status_code=500, detail=str(e))
95
96 @app.get("/")
97 async def read_index():
98     return FileResponse('static/index.html')

```

```
99
100 if __name__ == "__main__":
101     uvicorn.run(app, host="127.0.0.1", port=8000)
```

Ao finalizar o código acima, utilizei paradigmas de Javascript para conectar a API ao servidor na API, seguindo a configuração abaixo.

```
1      <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4     <meta charset="UTF-8">
5     <title>Psico EDU</title>
6     <style>
7         :root {
8             --bg: #0f172a;
9             --sidebar: #1e293b;
10            --accent-blue: #38bdf8;
11            --accent-green: #22c55e;
12            --text: #e2e8f0;
13            --bubble-ai: #334155;
14            --bubble-user: #0ea5e9;
15        }
16        body { margin: 0; background-color: var(--bg); color: var(--text); font-family: 'Segoe UI', Roboto, sans-serif; display: flex; height: 100vh; }
17
18        nav { width: 280px; background: var(--sidebar); border-right: 1px solid #334155; padding: 40px 25px; display: flex; flex-direction: column; justify-content: space-between; }
19        .sidebar-content h2 { color: var(--accent-blue); font-size: 1.5rem; margin-bottom: 5px; }
20        .sidebar-content p { color: #94a3b8; font-size: 0.9rem; line-height: 1.4; }
21
22        main { flex: 1; display: flex; flex-direction: column; padding: 50px; max-width: 900px; margin: auto; }
23        #chat-window { flex: 1; overflow-y: auto; margin-bottom: 30px; display: flex; flex-direction: column; gap: 20px; }
24
25        .bubble { padding: 20px; border-radius: 12px; font-size: 1.1rem; line-height: 1.6; max-width: 90%; }
26        .ai-bubble { background: var(--bubble-ai); border-left: 4px solid var(--accent-blue); align-self: flex-start; }
27        .user-bubble { background: var(--bubble-user); color: white; align-self: flex-end; border-radius: 12px 12px 0 12px; }
28
29        .controls { display: flex; gap: 15px; background: #1e293b;
```

```

padding: 20px; border-radius: 12px; border: 1px solid #334155; }
30     select { background: #0f172a; color: var(--accent-blue); border:
padding: 10px; cursor: pointer; border-radius: 6
px; }
31     input { flex: 1; background: transparent; border: none; color:
white; font-size: 1rem; outline: none; }
32     button { background: var(--accent-blue); border: none; color: #0
f172a; padding: 10px 25px; cursor: pointer; font-weight: bold; border
-radius: 6px; transition: 0.3s; }
33     button:hover { filter: brightness(1.2); }
34 </style>
35 </head>
36 <body>
37
38 <nav>
39     <div class="sidebar-content">
40         <h2>Psico EDU</h2>
41         <br>
42         <p><strong>CREATED BY:</strong><br>Jos   Mois s</p>
43         <p><strong>STUDENT:</strong><br>Computer Engineering</p>
44     </div>
45     <div style="font-size: 0.7rem; color: #64748b;">SYSTEM_STATUS:
ACTIVE</div>
46 </nav>
47
48 <main>
49     <div id="chat-window">
50         <div class="bubble ai-bubble">System initialized. Awaiting
Educational Psychology query...</div>
51     </div>
52
53     <div class="controls">
54         <select id="modelSelect">
55             <option value="modelo_3">QWEN_CORE_0.5B</option>
56             <option value="modelo_1">FLAN_T5</option>
57             <option value="modelo_2">BART_BASE</option>
58             <option value="modelo_4">GPT_NEO</option>
59         </select>
60         <input type="text" id="userInput" placeholder="RUN_QUERY >>"
onkeydown="if(event.key==='Enter') askAI()">
61         <button onclick="askAI()">EXECUTE</button>
62     </div>
63 </main>
64
65 <script>
66 async function askAI() {
67     const input = document.getElementById("userInput");

```

```

68     const chatWindow = document.getElementById("chat-window");
69     if (!input.value) return;
70
71     chatWindow.innerHTML += `<div class="bubble user-bubble">${input.
value}</div>`;
72     const query = input.value;
73     input.value = "";
74
75     const aiBubble = document.createElement("div");
76     aiBubble.className = "bubble ai-bubble";
77     chatWindow.appendChild(aiBubble);
78
79     try {
80         const res = await fetch('/chat', {
81             method: 'POST',
82             headers: { 'Content-Type': 'application/json' },
83             body: JSON.stringify({ query: query, modelo_id: document.
getElementById("modelSelect").value })
84         });
85         const data = await res.json();
86
87         let rawText = data.resposta.replace(/\n/g, '<br>');
88         let i = 0;
89         function typeWriter() {
90             if (i < rawText.length) {
91                 if (rawText.substr(i, 4) === '<br>') {
92                     aiBubble.innerHTML += '<br>';
93                     i += 4;
94                 } else {
95                     aiBubble.innerHTML += rawText.charAt(i);
96                     i++;
97                 }
98                 chatWindow.scrollTop = chatWindow.scrollHeight;
99                 setTimeout(typeWriter, 15);
100             }
101         }
102         typeWriter();
103     } catch {
104         aiBubble.innerHTML = "SYSTEM_FAILURE: Connection lost.";
105     }
106 }
107 </script>
108 </body>
109 </html>

```

Nenhuma parte desses procedimentos foi feito de forma solta, uma se liga a outra perfeitamente até se tornarem um conjunto particionado que resolve o problema real da

atividade em questão.

8 Aplicação com API funcionando com respostas direcionadas ao cliente

Todo o percurso até essa parte foi pensada em desenvolver o melhor trabalho para que os modelos trouxessem respostas de alto nível, sendo possível realizar o resultado muito em específico às perguntas. Começando pelo primeiro teste, utilizamos a pergunta enviada ao modelo Qwen 0.5B, que trouxe uma resposta completa em questão da Psicologia Educacional como ciência importante para diversas áreas. Para analisar, temos a seguinte Figura 9.

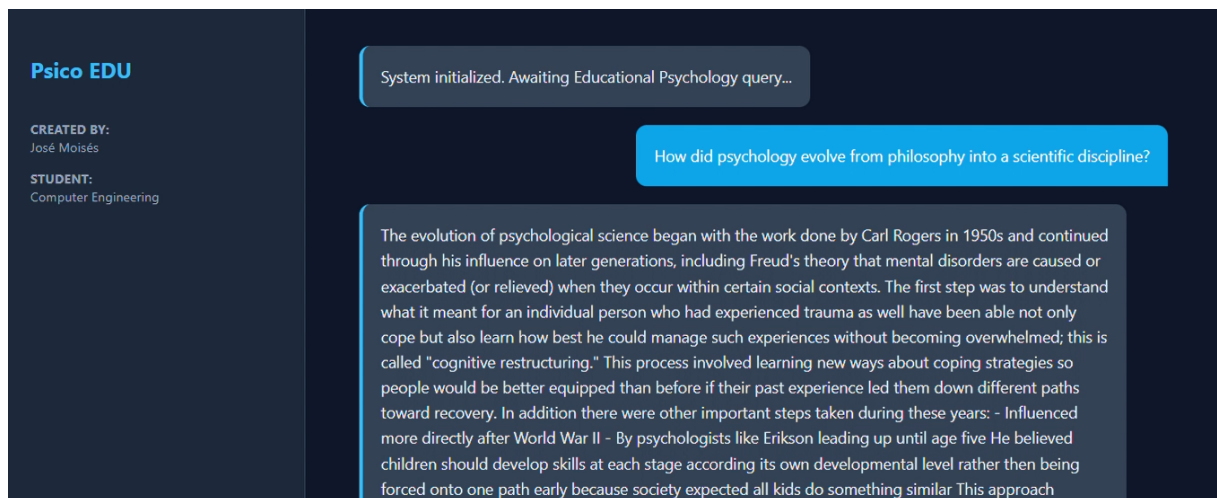


Figura 9: Resposta do Qwen 0.5B à pergunta do usuário.

Foi feito o estudo de que mesmo que a ideia de usar modelos menores tenha sido melhor pela melhor faixa de processamento em computadores menores, mas a resposta retornada ao usuário sendo menor, como visto na Figura10, quando o modelo Flant T5 Small responde simplesmente em pouco resultado e sendo bem simplório.

Mais abaixo, na Figura 11, vemos o funcionamento alucinado do BART Tiny, que deveria ter se saído melhor nos resultados, mas não retornou o devido teste realmente bem respondido.

O próximo modelo é o GPT-Neo-125M, que demonstrou uma resposta satisfatória, porém não excelente, visto que relacionou o processamento com dados externos ao dataset de treino, mantendo, contudo, um equilíbrio com as informações presentes na base de dados, conforme ilustrado na Figura 12.

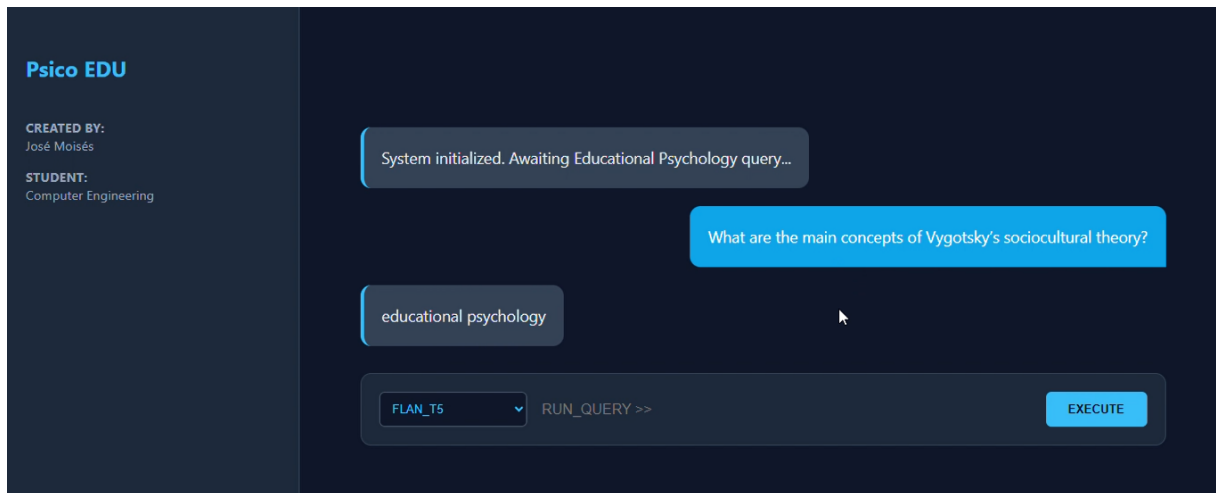


Figura 10: Resposta do Flan T5 Small à pergunta do usuário.

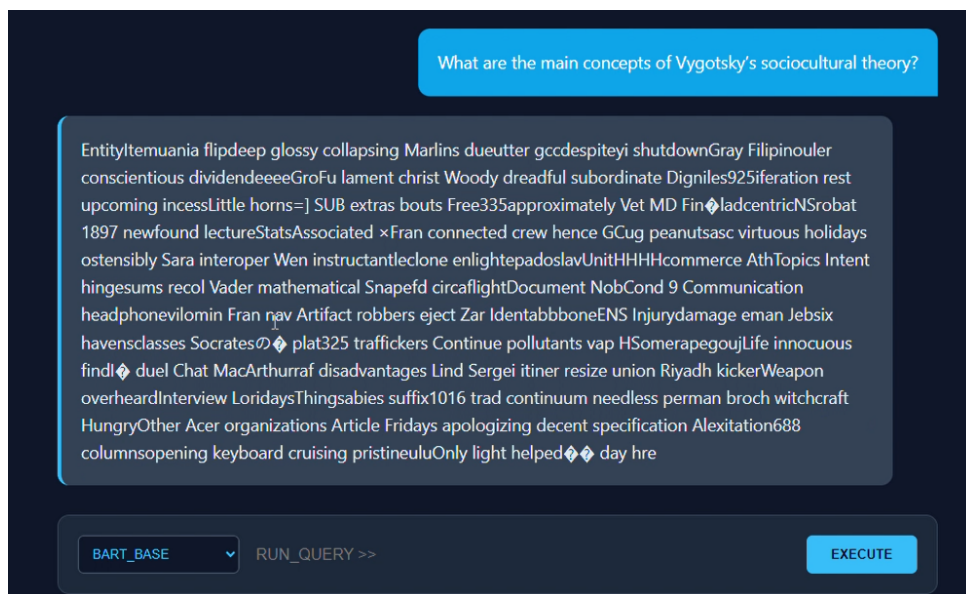


Figura 11: Resposta do Bart Tiny à pergunta do usuário.

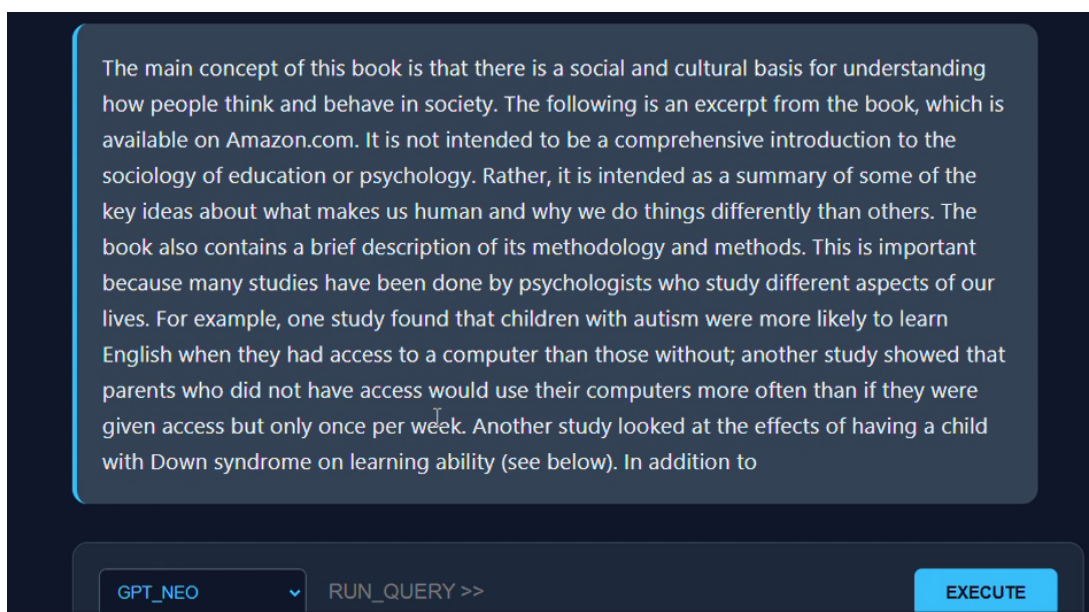


Figura 12: Resposta do GPT-Neo-125M à pergunta do usuário.

Portanto, foi seguido o processo de verificar como a API estava rodando no servidor, seguindo a ideia abaixo, na Figura 13 sobre que realmente funcionou a FastAPI para o sistema desenvolvido.

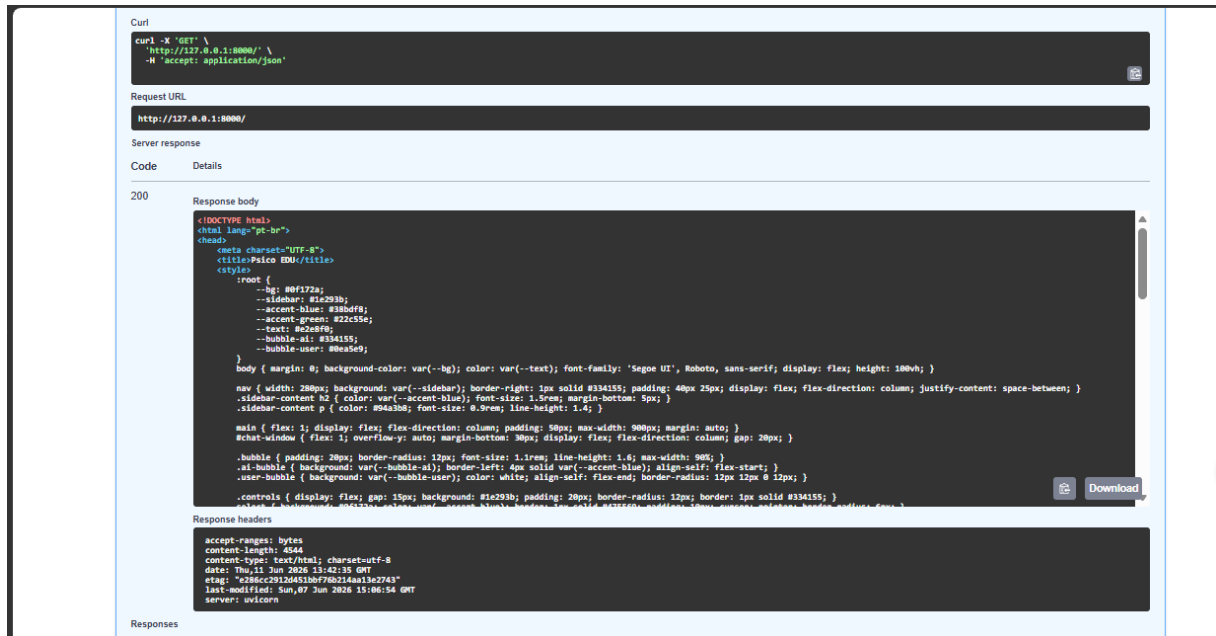


Figura 13: API funcionando

9 Considerações Finais

Todas as imagens e gráficos gerados foram consolidados na pasta /images para garantir a organização e a reprodutibilidade. Devido a limitações de hardware local, o treinamento foi realizado em ambiente Google Colab, com a subsequente migração e estruturação do projeto no VS Code, assegurando a integridade dos resultados apresentados. Este fluxo de trabalho demonstrou a viabilidade técnica de construir um agente especialista em Psicologia Educacional utilizando técnicas de RAG e Fine-tuning via LoRA.

Referências

- [1] Abhijit Chakraborty, Suddhasvatta Das, and Kevin Gary. Machine learning operations: A mapping study. In *World Congress in Computer Science, Computer Engineering & Applied Computing*, pages 3–21. Springer, 2024.
- [2] Danylo O Hanchuk and Serhiy O Semerikov. Implementing mlops practices for effective machine learning model deployment: A meta synthesis. In *AREdu*, pages 329–337, 2024.
- [3] Leif Sundberg and Jonny Holmström. Democratizing artificial intelligence: How no-code ai can leverage machine learning operations. *Business Horizons*, 66(6):777–788, 2023.
- [4] George B Thomas et al. Cálculo, vol. 1, 2002.